Auditoria do produto Zyphra Systems

Nov 2, 2026 · @Allan

Resumo executivo

A Zyphra tem um núcleo técnico forte e raro (gerir anúncios pelo WhatsApp com metodologia de curso codificada e ciclo anúncio → venda fechado), mas **ainda é um piloto de um cliente só**: 1 organização, 13 leads, 2 vendas enviadas à Meta, nenhuma campanha Google com cliques reais e o painel web sem publicação. Para cobrar de clientes faltam três coisas: fechar duas falhas de autorização entre empresas, publicar o painel e provar o sistema com um segundo cliente real.

Área	Veredito	Por quê
Atendente de IA no WhatsApp	Pronto	Usado com leads reais, travas de comportamento, passagem para humano
Malu — Meta Ads	Pronto	Criação, ativação e gestão reais, com confirmação e teto
Malu — Google Ads	Parcial	Construído e testado só em campanhas pausadas
Análise e otimização	Parcial	Meta automático diário; Google sob demanda; só recomenda
Atribuição e conversões	Parcial	Meta funcionando (15 eventos); Google sem venda aceita
Segurança	Risco alto	RLS completo, mas `outreach-router` e webhook da Evolution sem checagem de quem chama
Infraestrutura	Risco médio	VPS único, WhatsApp não oficial, plano gratuito do Supabase
Produto comercial	Não pronto	Painel não publicado, pool de IA vazio, implantação depende de suporte

Método: leitura do código (CRM e servidores MCP), consultas somente leitura ao banco de produção e ao painel de uso do Supabase em 02/11/2026. Nenhum arquivo, banco ou configuração foi alterado e nenhum teste invasivo foi feito.

1. Visão geral do produto

A plataforma é um CRM white-label com atendimento por IA no WhatsApp e um gestor de tráfego por IA (a Malu) que cria, ajusta e analisa campanhas Google e Meta. Hoje ela roda em produção para **1 organização, 1 usuário, 13 leads e 13 conversas** (banco ao vivo em 02/11/2026). Ou seja: o código é extenso, mas o uso real ainda é de piloto do próprio dono.

Tamanho medido: 92 Edge Functions (eram ~190 antes do refactor para roteadores), 263 tabelas, todas com RLS, 593 políticas, 329 migrations, 180 hooks de frontend, 65 ferramentas de IA, 26 tarefas agendadas ativas.

Módulo	Status	Evidência principal	Uso real comprovado
Atendimento por IA no WhatsApp (leads)	Implementado	`webchat-bot`, `evolution-webhook`, tabela `product_agents`	Sim: agente Natália atendeu leads reais das aulas
Gestor de tráfego IA (Malu) pelo WhatsApp	Implementado	`admin-agent-router`, `_shared/tools/impl/*`, `admin_agent_pending_actions`	Sim: campanhas Meta criadas e geridas; Google com testes
Criação de campanha Meta	Implementado	`_shared/meta-campaign-native.ts`, `meta-ads-guided-planner.ts`	Sim: campanha das aulas rodou
Criação de campanha Google (Pesquisa)	Implementado, pouco exercitado	`google-ads-guided-planner.ts`, `google-campaign-native.ts`	Só testes pausados; nenhuma campanha Google com cliques reais
Análise diária de campanhas Meta	Implementado	`marketing-campaigns-router` op `marketing_ai_analyze` (cron 09:00 UTC)	Sim: 5 diagnósticos abertos, 9 recomendações pendentes
Funil de vendas (pipeline)	Implementado	`pipeline_stages` (`is_won` / `is_lost`), `lead_stage_history`	Sim

Módulo	Status	Evidência principal	Uso real comprovado
Atribuição lead → anúncio	Implementado	`lead_attribution`, `attributeOrg`	7 leads atribuídos, todos por clique-para-WhatsApp
Envio de conversões à Meta (CAPI)	Implementado	`meta-ads-router` op `meta_capi_dispatcher`, `marketing_conversion_events`	15 eventos enviados, 2 deles Purchase
Envio de vendas ao Google	Implementado, nunca aceito em produção	`_shared/google-conversao-offline.ts`	1 evento, status `skipped`
Painel, relatórios, PDF	Implementado no código	`src/components/admin/relatorio/*`	Frontend não publicado
Cobrança (Cakto, Mercado Pago)	Implementado	webhooks e OAuth	Webhook ao vivo; Pix real recusado no teste
Agenda, quiz, formulários, voz IA, cadências	Implementado no código	rotas `/agendar`, `/q`, `/f`, `voice-*`	Não verificado nesta auditoria

Fluxo principal de um cliente: conectar o WhatsApp (Evolution) → configurar um atendente de IA por produto → conectar a conta Meta ou Google (OAuth) → pedir à Malu pelo WhatsApp "quero anunciar" → responder o assistente passo a passo → confirmar com código → o anúncio gera conversas → a Natália atende e move o lead no funil → venda marcada como Ganho volta para a Meta como Purchase e entra no retorno do anúncio.

Como os módulos se falam: tudo passa pelo Supabase. O WhatsApp chega no `evolution-webhook`, que chama o `webchat-bot`; se a conversa é do dono, o `webchat-bot` delega ao `admin-agent-router` (a Malu). Ações em anúncio saem por API nativa (Meta Graph, Google Ads REST) ou pelos servidores MCP no VPS. Vendas e etapas viram `journey_events`, que um cron de 5 em 5 minutos envia à Meta e ao Google.

Diferenciais frente a CRMs comuns: (1) operar anúncios por conversa de WhatsApp, com confirmação; (2) metodologias de cursos de tráfego (MTC, Sobral, Coloto) codificadas como regras determinísticas, não só como prompt; (3) ciclo fechado anúncio → conversa

→ venda → conversão de volta para a plataforma; (4) atendente de IA com travas comportamentais (não insistir, não repetir nome, mover etapa ao falar preço).

2. Gestor de tráfego com inteligência artificial

O gestor de tráfego (Malu) é a parte mais madura e mais trabalhada do código. Mas ele é mais um **assistente guiado por regras** do que um agente autônomo: a IA conversa e extrai dados; o que cria, muda ou gasta dinheiro passa por código determinístico e por confirmação do dono. Isso é uma escolha deliberada, registrada no código depois de casos reais em que a IA "disse que fez" sem fazer.

Status: implementado (Meta com uso real; Google com testes, sem campanha real com cliques).

Como funciona

Aspecto	O que o código faz	Evidência
Modelo de IA	`gpt-5-mini` (OpenAI) na conversa e nos planejadores; `whisper-1` / `gpt-4o-transcribe` para áudio; `gpt-4o-mini` em pontos menores. Resolução por organização: chave própria → plano → pool da plataforma	`_shared/ai-call.ts` (`aiChat`), `_shared/ai-router.ts` (`resolveAIConfig`), `callAdminAI` em `admin-agent-router`
Conhecimento dos cursos ("RAG")	**Não é RAG vetorial.** Os cursos viraram frameworks JSON (regras PC/EL/MA/CO, OT, NL) e regras em TypeScript. A busca em 16 módulos + 143 estudos de nicho + 53 playbooks é léxica (pontuação por palavras)	`mcp_server/tools/busca_conhecimento.py` (`_score_google`), `_shared/google-ads-mtc-regras.ts` , `_shared/regras-sobral-meta.ts`
Uso da busca de conhecimento	A ferramenta `buscar_conhecimento_estruturado_ads` existe, mas **não está na lista de ferramentas que a Malu enxerga** (`ADS_TOOL_NAMES`). O conhecimento entra nos planejadores (verba por nicho, aviso de risco), protegido pela trava `nichoEhDoMesmoRamo`	`admin-agent-router/index.ts` linhas 83-156
Entender o negócio	Assistente passo a passo (`beginner_wizard`): tipo de negócio → busca ativa → endereço/raio → descrição → orçamento → destino. Persona por produto com campos críticos confirmados pelo dono	`admin_agent_pending_actions` , `org_ad_personas` , `_shared/ad-persona-store.ts`
Google ou Meta	Pela pergunta "o cliente procura ativamente?" (passo `busca_ativa`); sem site, o Google oferece página de captura do CRM ou migra para Meta/WhatsApp	`admin-agent-router` (passos do assistente)
Orçamento	Faixa por tipo de negócio, ajustada por nicho do mesmo ramo (teto 1,5×); Google confere a regra dos 10 cliques por dia com o CPC pesquisado	`faixaGenericaOrcamentoAssistente` , `limiteEstruturaPorCliques`

Aspecto	O que o código faz	Evidência
Copy e criativos	Google: IA gera títulos/descrições; código corta promessas não ditas pelo dono, números inventados e palavra solta; Título 1 = palavra-chave. Meta: até 3 fotos do dono + várias copies. **Não gera imagem**	`consertarCopy`, `promessasNaoSustentadas`, `meta-campaign-native.ts`
Criar campanha	Google: um `googleAds:mutate` atômico. Meta: Graph API direto. Tudo nasce **pausado**	`google-campaign-native.ts`, `meta-campaign-native.ts`
Gerir campanha	Pausar, reativar, mudar verba, remover (campanha ou um conjunto/grupo); negativas; teste A/B de anúncio Google	`preparar_gestao_campanha`, `confirmar_acao_pendente_ads.executarGestao`
Memória e continuidade	Histórico da conversa, persona por produto, rascunhos de campanha (7 dias, até 3), pedidos pendentes com validade de 30 min, log de ferramentas executadas	`webchat_messages`, `org_ad_personas`, `draft_parked`, `agent_tool_executions`

Onde exige confirmação humana

1. Criar a campanha (nasce pausada, sem gasto).
2. Ativar a campanha (aqui começa o gasto).
3. Pausar, reativar, mudar verba ou remover.
4. Bloquear termos negativos sugeridos.
5. Criar anúncio de teste no Google (exceção: nasce ativo e pausa o campeão no mesmo comando).

A confirmação é detectada **no código, não pela IA**, e vale para o pedido gravado. Ela aceita o código ou uma afirmação curta e clara ("sim", "confirmo"). Teto de R$ 500/dia por campanha (`TETO_POR_CAMPANHA`) e aviso quando a verba sobe mais de 20%.

Ferramentas que a Malu pode usar

39 ferramentas liberadas, entre elas: `iniciar_assistente_campanha_simplificado`, `listar_campanhas`, `preparar_gestao_campanha`, `confirmar_acao_pendente_ads`, `iniciar_diagnostico_campanha`, `revisar_termos_de_pesquisa_google_ads`, `preparar_teste_anuncio_google`, `plano_otimizacao_google`, `oportunidades_de_verba_google`, `verificar_rastreamento_site`, `saude_do_rastreamento`, `consultar_painel`, `conectar_google_ads_cliente`. Há 65 ferramentas no registro; as demais são internas aos planejadores ou estão fora de uso.

Limitações identificadas

- **Google sem prova real:** nenhuma campanha Google da Malu rodou com cliques. Os itens de otimização Google (verba, plano, teste A/B, indicador, negativas semanais) nunca foram vistos funcionando com dados reais.

- **Google só Pesquisa.** Sem Display, Performance Max, YouTube (exceto um lembrete experimental) ou Shopping.
- **Meta sem geração de criativo.** Depende de foto do dono; sem vídeo hoje.
- **App Google OAuth não verificado:** tela "app não verificado" e limite de 100 usuários; nível Explorer (sem Keyword Planner na conta do cliente — a pesquisa de palavras usa a conta fixa da agência).
- **Token Meta de 60 dias sem renovação automática** encontrada; o cliente precisa reconectar.
- **Funções gigantes:** `admin-agent-router/index.ts` concentra o fluxo inteiro (milhares de linhas, prompt + desvios + estados), o que torna cada mudança arriscada.

O que ainda precisa ser validado

- Ciclo completo Google com cliente real: conectar → criar → ativar → 7 dias de dados → diagnóstico.
- Reconexão Meta após expirar o token.
- Comportamento com um segundo cliente (hoje só há 1 organização).

3. Análise e otimização de campanhas

Não existe uma regra "analisa só depois de 7 dias" no código. O que existe são travas de volume e de fase de aprendizado, diferentes por plataforma. O "7 dias" que aparece no código serve para outra coisa: um diagnóstico ignorado ou resolvido há menos de 7 dias não volta a aparecer.

Status: Meta implementado e rodando todo dia; Google implementado sob demanda, sem dado real para validar.

Meta: motor automático diário

Roda às 09:00 UTC (`marketing-ai-analyze`, op `marketing_ai_analyze` em `marketing-campaigns-router`), com janela padrão de 14 dias (1 a 90). Lê gasto, impressões, cliques, CTR, CPM, frequência, conversas e leads **por anúncio** (`marketing_insights_daily`, `entity_type='ad'`) e soma para conjunto e campanha. Conjuntos em fase de aprendizado são pulados.

Regra	Nível	Gatilho	Saída
1	Anúncio	3 dias sem lead e gasto ⩾ 2× o custo por lead médio da conta	Recomenda pausar o anúncio

Regra	Nível	Gatilho	Saída
2	Campanha	Custo por lead dos últimos 3 dias 30% acima da média da conta	Recomenda reduzir verba
3	Conjunto	Frequência de 7 dias > 3,0 e CTR caindo 3 dias seguidos (Sobral)	Achado de fadiga
4	Conjunto	Entre anúncios do mesmo conjunto, custo por lead do pior > 50% acima do melhor	Recomenda pausar o pior criativo
5	Campanha	CTR < 70% do CTR médio da conta	Só diagnóstico, sem ação
6	Campanha	Leads que viram "perdido" no funil acima da média	Achado de qualidade de lead (usa o CRM)
7	Conjunto	Custo alto sustentado e já há conversão personalizada criada	Recomenda migrar a otimização
8	Conjunto	Pior público ≥ 1,5× o melhor, 500+ impressões (OT05)	Recomenda trocar o público
9	Campanha	Menos de 70% de quem clicou mandou mensagem, com 15+ cliques	Achado de perda entre clique e conversa

Há também o diagnóstico guiado (na tela e pela Malu), que pergunta ticket, vendas e quanto aceita pagar por cliente e aplica OT01-OT04, NL03, NL11, NL12 e MET06.

Google: diagnóstico sob demanda e relatório semanal

- **"Nunca vendeu":** só com 100+ cliques; percorre 6 pilares em ordem (técnico → público → anúncio → site → comercial → mercado) e para no primeiro problema (`_shared/google-ads-mtc-regras.ts`).
- **"Quero vender mais":** 7 passos com teto de custo por venda, idade/gênero/cidades em 90 dias, Ponto A → B.
- **Anúncio campeão:** compara anúncios com 50+ cliques (`google-ads-ad-champion.ts`).
- **Termos de pesquisa:** IA classifica comprador/curioso/irrelevante e propõe negativas (`google-ads-search-terms-cleanup.ts`).
- **Relatório semanal** (segunda) com oportunidades de verba, testes e negativas.

Respostas diretas

Pergunta	Resposta
Analisa campanha, conjunto e anúncio?	Sim, na Meta. No Google: campanha, grupo, anúncio e termo de pesquisa
Analisa cada criativo?	Sim por métricas (Regras 1 e 4). Não analisa a imagem em si; mede taxa de retenção de vídeo só na tela
Compara cliques, conversas, leads e vendas?	Sim: Regra 9 (clique → conversa), Regra 6 (lead → perdido), retorno com vendas marcadas como Ganho
Usa CPL, CPC, CTR, CPM?	Sim, todos aparecem; a tela troca as siglas por "custo por contato" etc.
Sugere novos criativos e copies?	Google: reescreve o anúncio campeão mudando uma hipótese. Meta: só orienta e troca a foto enviada pelo dono
Executa ou só recomenda?	**Só recomenda por padrão.** O dono aplica no painel (`marketing_ai_execute`) ou confirma pela Malu. Regras automáticas existem (`marketing_ai_rules` , modo `auto`), mas **nenhuma está cadastrada**
Compara períodos?	Parcial: últimos 3 dias contra a média da conta; testes antes × depois com 30 dias. Não há "este mês contra o anterior" no motor
Monitora sozinho?	Meta sim (diário). Google só quando o dono pede ou no relatório semanal

Limitações

- **Venda da Meta não é usada como prova:** `SALES_VERIFIED = false` no motor, então regras por compra ficam desligadas. O retorno usa as vendas marcadas no funil, o que depende do dono marcar Ganho com valor.
- Com a verba atual (R$ 12/dia, anúncios parados desde 15/10) quase nenhuma regra atinge o volume mínimo; o motor gera poucos achados.
- As 9 recomendações atuais estão todas `pending` : nunca foram aplicadas, então o efeito real das recomendações não foi medido.

- Limiares como 1,5×, 30% e 70% são heurísticas do código ou do curso, sem calibração com dados de vários clientes.

4. Integração Meta Ads e Google Ads

As duas integrações existem por dois caminhos: **conta própria do cliente (OAuth, API nativa)**, que é o caminho multi-cliente de verdade, e **modo agência (servidores MCP no VPS)**, preso às credenciais do Allan. Hoje a organização de teste tem as duas conexões ativas no Google e a OAuth ativa na Meta.

O ciclo fechado está construído e funcionou na Meta: 15 eventos aceitos (2 vendas). No Google, nenhuma venda foi aceita até hoje (1 evento, `skipped` porque a venda foi cancelada antes do envio).

Item	Meta Ads	Google Ads
Conexão da conta	OAuth (`meta-oauth-start/callback/finalize`), token de longa duração (~60 dias) criptografado. **Sem renovação automática encontrada**	OAuth (`google-ads-oauth-start/callback`), refresh token; app em produção mas **não verificado** (limite de 100 usuários, nível Explorer). Conexão que cai avisa o dono (`google-conexao-saude.ts`)
Criar e gerir campanhas	Implementado (Graph API, `meta-campaign-native.ts` , `meta-ads-conta.ts`)	Implementado (REST atômico, `google-campaign-native.ts` , 17 leituras espelhadas em `google-ads-conta.ts`)
Ler métricas	Sync por anúncio de hora em hora (`marketing-sync-meta-tick`), também por plataforma Facebook/Instagram	Sync a cada 15 min (`marketing-sync-google-tick`)
Origem do lead	ID do anúncio do clique-para-WhatsApp (`externalAdReply.sourceID` , `referral.source_id`), UTM `nome|id` , fbclid	gclid em formulário, funil e SDK de rastreamento; código "(ref. XXXXX)" no botão de WhatsApp do site (`whatsapp_click_refs`)
Onde fica guardado	`lead_attribution` (método `ctwa` , `utm_exact` , `utm_fuzzy` , `fbclid` ...), `leads.ad_external_id`	`leads` / `tracking_events` com gclid

Item	Meta Ads	Google Ads
Eventos de conversão	CAPI: etapa do funil vira evento (`Etapa_Novo_Contato`, `Etapa_Valor_Enviado_Aguardando`), `LeadDesqualificado`, `Purchase`	Data Manager API: "CRM - Venda fechada" (conversão secundária criada sozinha)
Lead qualificado	Sim, por etapa do funil	Não envia lead, só venda
Venda e valor	`Purchase` com `deal_value` (ou valor de Cakto, Hotmart, link de pagamento)	Venda com valor; cancelamento por ajuste de conversão; correção de valor reenviando o mesmo ID
Deduplicação	`dedup_key` estável, único por organização + plataforma (`stableDedupKey`)	Chave `google:venda:<id do negócio>`
Erro e reenvio	Status `pending` / `sent` / `skipped` /falha registrado; `force_retry` manual	Pendentes reenviados por 60 dias; conversão recém-criada não gasta tentativa; sem gclid e sem termos aceitos fica pendente e avisa o dono
Registro aceito/rejeitado	`marketing_conversion_events` (status + `error_message`)	A mesma tabela

Como o ciclo fecha

1. O anúncio de clique-para-WhatsApp traz o ID do anúncio no webhook.
2. `attributeOrg` liga o lead ao anúncio, ao conjunto e à campanha.
3. O atendente move o lead no funil; cada etapa vira `journey_events`.
4. O cron `meta-sync-won-deals` (a cada 5 min) envia etapa, desqualificação e venda à Meta, e a venda ao Google.
5. A função `rpc_marketing_campaign_performance` soma gasto por anúncio e vendas Ganho atribuídas para calcular o retorno.

Limitações

- **Lead da Meta sem gclid não vai ao Google** (correto), e lead do Google só é rastreado se o site tiver o SDK ou o formulário do CRM.
- **Anúncio criado direto no Gerenciador sem parâmetros de URL** não é atribuído (só fbclid); o sistema avisa (`notifyAttributionGap`).

- **Modo agência não é multi-cliente:** as campanhas de agência caem numa conta Google pessoal do Allan, sem MCC operando.
- **WhatsApp oficial (Meta Cloud) bloqueado:** o número está verificado, mas o registro exige CNPJ. Hoje o atendimento roda na Evolution (não oficial).

O que precisa ser validado

- Uma venda real aceita pelo Google (gclid → Data Manager → conversão visível na conta).
- Se os eventos customizados de etapa (`Etapa_*`) são de fato usados pela Meta para otimizar, ou só registrados.

5. CRM e rastreamento dos leads

O CRM consegue ligar uma venda ao anúncio de origem e calcular retorno com vendas reais. Mas isso só vale para leads que chegam por clique-para-WhatsApp ou por formulário/site rastreado, e só quando alguém marca a venda como Ganho com valor. Dos 13 leads atuais, 7 têm anúncio de origem identificado.

Funcionalidade	Status	Como funciona	Evidência
Entrada do lead	Implementado	WhatsApp (Evolution e oficial), Instagram Direct, chat do site, formulário, funil, quiz, agenda, ligação de voz, webhooks de loja/pagamento, cadastro manual	`evolution-webhook`, `instagram-webhook`, `form-submit`, `funnel-submit`, `voice-inbound-lead`, `nuvemshop-webhook`
Origem	Implementado	ID do anúncio, UTM, gclid/fbclid, plataforma Facebook × Instagram	`lead_attribution`, `leads.ad_external_id`, `leads.metadata.plataforma_anuncio`, `_shared/origem-anuncio.ts`
Jornada	Implementado	Cada etapa vira evento; tela de Jornada com funil cumulativo (corrigido em 01/11 para não passar de 100%)	`journey_events`, `lead_stage_history`, `JourneyDashboard.tsx`
Interações	Implementado	Todas as mensagens, quem atendeu (bot ou humano), transferências	`webchat_messages`, `webchat_assignment_events`, `agent_tool_executions`

Funcionalidade	Status	Como funciona	Evidência
Funil por produto	Implementado	Cada produto tem seu funil; lead do WhatsApp entra na 1ª etapa quando a conversa identifica o produto	`pipeline_stages.product_id`, `linkLeadToProductPipeline`
Ganho e desqualificado	Implementado	Etapa com `is_won` / `is_lost`; botões Ganhou/Perdeu; motivo da perda gravado só quando o lead diz	`pipeline_stages`, `leads.loss_reason`
Valor da venda	Implementado	`leads.deal_value`, tabela `deals`, vendas de checkout em `commerce_sales`	`rpc_marketing_campaign_performance`
Envio à Meta e ao Google	Implementado	Ver seção 4	`meta-ads-router` (`meta_sync_won_deals`), `google-conversao-offline.ts`
Venda → campanha, conjunto e anúncio	Implementado	Atribuição é por anúncio; conjunto e campanha vêm da hierarquia sincronizada	`attributeOrg`, `marketing_ads`
Retorno (ROAS) com venda real	Implementado	Gasto por anúncio ÷ vendas Ganho atribuídas, sem duplicar venda confirmada de checkout; aparece no Painel, na Malu e no relatório PDF	migration `20261009180000`, `RelatorioPdfSection.tsx`

Limitações

- **O retorno depende de disciplina manual:** se o vendedor não marcar Ganho com valor, a venda não existe para o sistema. Não há conciliação automática com nota fiscal ou extrato.
- **ROI completo não existe:** há retorno sobre anúncio (receita ÷ gasto), mas não há custo do produto, margem ou custo da IA e do WhatsApp por cliente no cálculo. O custo do WhatsApp oficial tem um card próprio, nunca testado ao vivo.
- **Lead orgânico ou de indicação** fica sem origem (método `unresolved`), o que é esperado.

- Com 13 leads e 2 vendas, os números de retorno ainda não têm valor estatístico.

6. Agente de IA pelo WhatsApp do proprietário

O WhatsApp é uma interface de controle, não só de consulta. Pelo WhatsApp o dono cria campanha, ativa, pausa, muda verba, remove, bloqueia termos e testa anúncio, sempre com confirmação. Na web, o botão "Perguntar à Malu" (01/11) é só leitura.

Status: implementado e usado no dia a dia pelo Allan.

Aspecto	Como funciona	Evidência
Integração	Mensagem chega no `evolution-webhook` → `webchat-bot` vê que o agente da conversa é do tipo `admin` → delega ao `admin-agent-router`. Aceita texto e áudio (transcrição)	`webchat-bot/index.ts` (verificação de "admin takeover"), `opAdminAgentHandleInboundComAudio`
Quem é o dono e a empresa	Empresa: vem da linha da conversa (`webchat_conversations.organization_id`). Pessoa: o **número de telefone** de quem mandou é casado com um usuário autorizado, e o acesso a tráfego é conferido na hora	`resolveTrafficManagementWhatsAppSender`, `checkTrafficManagementAccess`
Consultar CRM e campanhas	Painel do mês, funil ("onde os leads param?"), lista de campanhas Google + Meta, métricas por campanha e plataforma	`consultar_painel`, `listar_campanhas`, `_shared/funil-conversas.ts`, `detalhar_campanha_meta`
Processar comandos	Comandos claros (listar, cancelar, confirmar, rascunhos, pausar/reativar) são resolvidos **no código antes da IA**; o resto vai para a IA com as 39 ferramentas liberadas	desvios determinísticos em `opAdminAgentHandleInbound`
Ações	Criar campanha (Meta e Google), ativar, pausar, reativar, mudar verba, remover, pausar um conjunto, negativar termos, trocar foto do anúncio, teste A/B Google, conectar conta Google, gerar link de pagamento	seção 2
Confirmar gasto	Pedido gravado em `admin_agent_pending_actions` com código e validade de 30 min; só o "confirmar" (ou código) executa; teto de R$ 500/dia por campanha	`confirmar_acao_pendente_ads`, `TETO_POR_CAMPANHA`
Contexto	Histórico da conversa, pedidos abertos, rascunhos, persona do produto; volta ao assunto depois de uma pergunta paralela	`redirecionarForaDoAssunto`, `draft_parked`
Mensagens proativas	Resumo da manhã (8h), relatório semanal, alertas de verba e de conexão caída, aviso de lead quente	op `admin_agent_summary`, `admin-agent-alerts`
Isolamento entre empresas	Toda consulta filtra pela empresa da conversa; a chamada direta ao `admin-agent-router` passou a exigir chave de serviço ou usuário da própria empresa (falha fechada em 01/11, commit `4eb4b04`)	`quemChama`

Riscos específicos deste canal

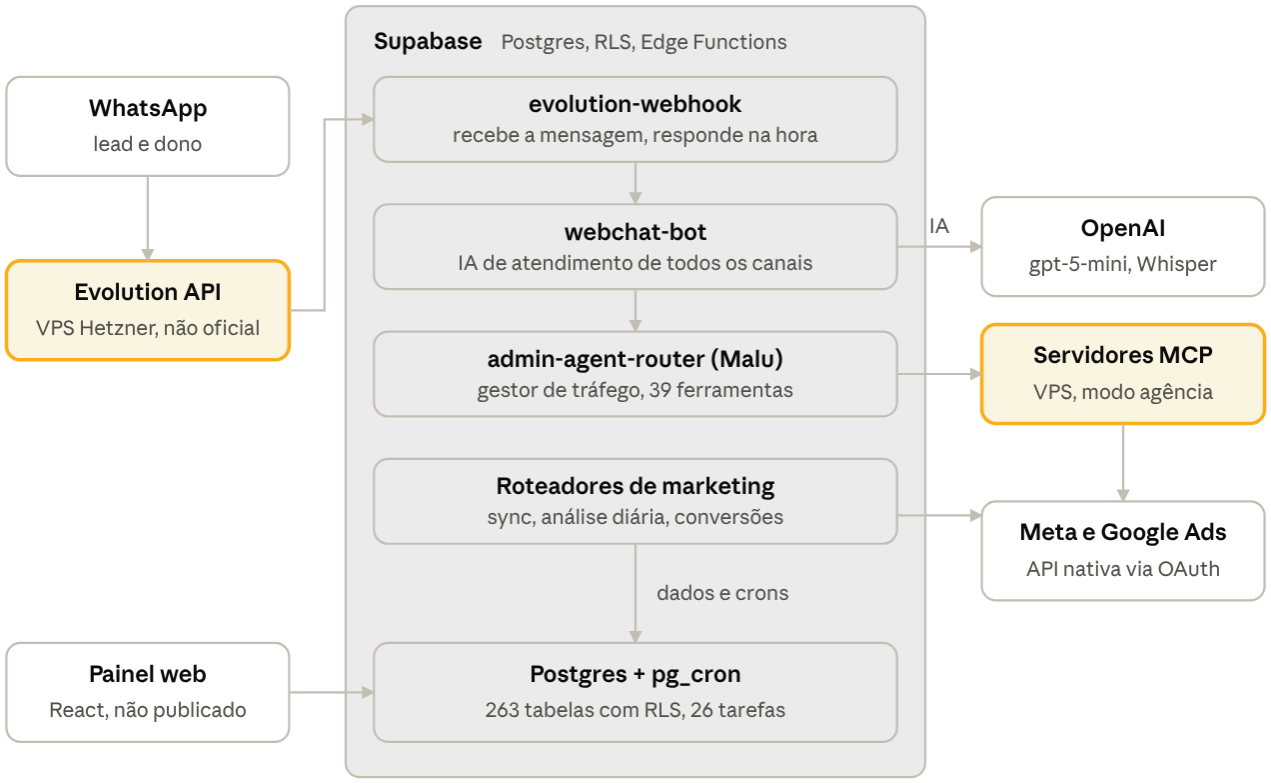
- **Identidade pelo número de telefone, num webhook sem autenticação.** O `evolution-webhook` é público (`verify_jwt = false`) e não confere nenhum segredo da Evolution; ele só procura a instância pelo nome. Em teoria, quem souber a URL, o nome da instância e o número do dono poderia forjar mensagens "do dono". Como a confirmação aceita "confirmo" sem o código, a mesma pessoa poderia confirmar. **Precisa de teste controlado para confirmar** (ver seção 8).
- **Um só número de WhatsApp não oficial** atende o dono e os leads; se a Evolution cair ou o número for banido, a Malu e o atendimento param juntos.
- Uma resposta da Malu leva de 28 a 90 segundos (medido nas sessões de teste), o que pode parecer travado para um cliente.

7. Arquitetura e tecnologia

A arquitetura é simples de operar (quase tudo no Supabase), mas concentra risco em três dependências externas: o VPS Hetzner (WhatsApp e MCPs), a OpenAI (toda a IA) e o plano gratuito do Supabase.

Tudo passa pelo Supabase; WhatsApp e modo agência dependem de um único VPS

Em laranja: pontos únicos de falha fora do Supabase



arquitetura · Supabase no centro, VPS Hetzner e OpenAI como dependências externas

A mensagem do WhatsApp atravessa três funções em sequência; anúncios saem por API nativa (OAuth) ou pelos MCPs no VPS.

Camada	Tecnologia	Observação
Frontend	Vite + React 18 + TypeScript, shadcn/Radix, Tailwind, React Query	**Não hospedado**; roda só em `localhost:8080`. Redesign das fases 0-4 feito localmente
Backend	92 Edge Functions Deno no Supabase, agrupadas em roteadores por `op`	Arquivos muito grandes (`admin-agent-router` , `marketing-campaigns-router` e webchat-bot, de 6.500 a 7.100 linhas cada)
Banco	Postgres com 263 tabelas, RLS em todas, 593 políticas, pgvector para memória de lead	`has_role` , `get_user_organization` como base das políticas
Autenticação	Supabase Auth, login Google nativo, e-mail por Resend com hook de envio	Primeiro Super Admin via `/setup`
Multi-tenancy	`organization_id` em cada tabela + RLS; papéis em `user_roles`	Testado com **uma** organização só
Filas e assíncrono	`EdgeRuntime.waitUntil` + fila de rede de segurança (`process_inbound_queue`), pgmq para e-mail, 26 tarefas pg_cron (várias com guarda que evita chamada quando não há trabalho)	Alguns crons e segredos do vault **não estão em migration** (criados à mão)
IA	OpenAI (gpt-5-mini, gpt-4o-mini, Whisper, embeddings); configuração também prevê Gemini	Pool de chaves da plataforma **vazio**: cliente sem chave própria fica sem IA
Integrações	Meta Graph e Marketing API, Google Ads REST + Data Manager, Evolution, Instagram, Resend, Cakto, Mercado Pago, Hotmart, Nuvemshop, Bling, xAI (voz)	
Infraestrutura	Supabase Cloud (plano gratuito), VPS Hetzner com Docker Swarm (Evolution, Postgres da Evolution, 2 MCPs com 3 réplicas cada, watchdog)	Deploy do VPS é manual por `scp` (pasta não versionada)

Pontos únicos de falha

1. **VPS Hetzner:** WhatsApp (dono e leads) e modo agência ficam no mesmo servidor.
2. **Número de WhatsApp não oficial:** pode ser banido; o oficial está travado por falta de CNPJ.
3. **OpenAI:** a configuração permite trocar de provedor, mas na prática só há uma chave (da própria organização).
4. **Plano gratuito do Supabase:** período de tolerância acabou em 17/10; se a cota estourar, o projeto pode ser limitado (uso atual: 6% das chamadas).
5. **Configuração fora do Git:** crons e segredos criados à mão; um ambiente novo não sobe só com as migrations.

8. Segurança e confiabilidade

O banco está bem protegido no papel (RLS em todas as 263 tabelas), mas **o risco real está nas Edge Functions**: elas usam a chave de serviço, que ignora o RLS, e algumas confiam na empresa informada no próprio pedido. Uma falha desse tipo já foi encontrada e fechada na Malu em 01/11; esta análise achou outra no mesmo padrão. Tudo abaixo vem de leitura do código; nenhum teste invasivo foi feito.

Achados por gravidade

Gravidade	Achado	Evidência	O que exigiria teste
Alta	`outreach-router` não confere quem chama: aceita `organization_id`, `lead_ids` e `agent_id` do corpo do pedido e usa a chave de serviço. Qualquer pessoa com a chave pública do app (que fica no frontend) e os IDs certos poderia fazer a IA mandar WhatsApp para leads de outra empresa e devolver a conversa ao bot. Os IDs são UUID (difíceis de adivinhar), o que reduz, mas não elimina, o risco	`outreach-router/index.ts` (`Deno.serve` sem checagem de usuário), `_shared/outreach-core.ts`	Chamar a função com a chave pública e IDs de uma segunda organização de teste

Gravidade	Achado	Evidência	O que exigiria teste
Alta	`evolution-webhook` é público e não confere segredo da Evolution; o dono é reconhecido pelo número de telefone do payload e a confirmação aceita "confirmo" sem código. Quem souber URL, nome da instância e número do dono poderia comandar a Malu	`config.toml` (`verify_jwt = false`), busca de instância por nome em `evolution-webhook` , `resolveTrafficManagementWhatsAppSender`	Enviar payload forjado a um ambiente de teste e ver se a Malu responde
Média	A fila de rede de segurança decodifica o JWT sem verificar a assinatura (função sem `verify_jwt`); um token forjado com `role: service_role` passaria. Impacto limitado: só reprocessa mensagens paradas	`evolution-webhook` , op `process_inbound_queue`	Chamar com token forjado em ambiente de teste
Média	Nome da instância vindo do payload é interpolado num filtro `.or(...)` do PostgREST	`evolution-webhook` (busca em `evolution_instances`)	Teste de injeção de filtro
Média	Migration antiga cria um Super Admin com senha fixa em banco novo; precisa confirmar que foi neutralizado em produção	`20260506194031_*.sql` , `useSuperAdminFirstAccess.ts`	Conferir a conta no Auth
Baixa	Algumas funções devolvem `String(err)` na resposta (texto técnico do erro)	`outreach-router` , outras	Revisão de código

O que está bem resolvido

- **Credenciais:** segredos ficam nos secrets do Supabase; tokens Meta e Google ficam criptografados no banco (`meta-crypto.ts`). Não encontrei segredo no código nesta análise.

- **Webhooks de pagamento e loja** verificam assinatura (Nuvemshop por HMAC; Resend e Auth Hook por Standard Webhooks).
- **Ações da IA que gastam dinheiro** são executadas só pelo código, a partir de um pedido gravado, nunca porque a IA "disse" que fez. Campanha nasce pausada, teto de R$ 500/dia, aviso de aumento acima de 20%.
- **Proteção contra outro robô** (`bot-loop-guard.ts`) e trava de conversa com identidade (evita duas respostas ao mesmo tempo).
- **Registro de operações:** `agent_tool_executions` (entrada, saída, sucesso), `admin_agent_messages` , `marketing_conversion_events` , log de auditoria no MCP.

Prompt injection

O texto do lead entra no prompt do atendente. A defesa principal é estrutural: o atendente de lead tem ferramentas limitadas e as ações de tráfego só existem na conversa do dono. **Não há filtro específico de injeção.** Precisa de teste: um lead tentar fazer o atendente gerar link de pagamento com outro valor, mover etapa ou revelar instruções internas.

Recomendação

Antes do segundo cliente: auditar **todas** as funções que recebem `organization_id` no corpo do pedido (o padrão já apareceu duas vezes) e exigir segredo no webhook da Evolution.

9. Custos e escalabilidade

O custo que mais cresce com clientes é o de **tokens de entrada da IA**: cada resposta do atendente envia cerca de 7.800 tokens e cada resposta da Malu cerca de 11.200, para devolver menos de 100. O prompt grande (instruções, regras, histórico, lista de ferramentas) é o que pesa, não a resposta.

Consumo de IA medido (últimos 30 dias, `ai_usage_logs`)

Fluxo	Chamadas	Tokens de entrada por chamada	Tokens de saída por chamada
Atendente de leads (`webchat-bot`)	151	~7.800	~48
Malu (`admin-agent-router`)	100	~11.200	~69
Planejador Google	97	~1.260	~355

Fluxo	Chamadas	Tokens de entrada por chamada	Tokens de saída por chamada
Assistente de campanha (`wizard-fallback`)	86	~1.250	~98
Planejador Meta	32	~2.030	~380

Criar **uma** campanha Google dispara várias chamadas encadeadas (brief, grupos e anúncios, classificação de intenção, revisão de negativas, conserto de copy, extensões). O número exato por plano não é registrado por plano, só por função.

Respostas diretas

Pergunta	Resposta
Como o "RAG" afeta tokens?	Pouco: a busca é léxica e o texto entra truncado (brief 2.500 caracteres, framework de copy 1.500-2.000). O peso está no prompt fixo do agente
Há cache?	Não identifiquei cache deliberado de prompt nem de resposta da IA. No frontend, React Query guarda dados por 5 minutos
Tarefas redundantes?	Sim: cada mensagem de WhatsApp atravessa 3 funções separadas (webhook → atendente → Malu); a parte 1 (envio direto, 01/11) tirou as chamadas de envio; a parte 2 (juntar as 3) está planejada
Erros e novas tentativas	Brief da IA com 2 tentativas; MCP com 3 tentativas só para erro temporário; fila de entrada até 5 tentativas; vendas Google pendentes por 60 dias. Uma tentativa que trava pode gerar reprocessamento duplicado (já aconteceu; limite subiu para 170 s)
Serviços que geram custo	OpenAI (tokens), Supabase (chamadas de função, banco, egress), VPS Hetzner, WhatsApp oficial quando ativado (janela paga ~R$ 0,04), Resend (e-mail), xAI (voz), domínio e hospedagem do frontend. As APIs de anúncio não cobram

Onde aperta com 10, 50, 100 e 500 clientes

Clientes	O que tende a apertar primeiro
10	Pool de chaves de IA vazio: cada cliente precisa de chave própria, ou a plataforma precisa abastecer o pool. Limite de 100 usuários do app Google não verificado ainda não pesa
50	Cota de 500 mil chamadas/mês do plano gratuito do Supabase (o tráfego de mensagens soma ao custo fixo de ~85 mil dos crons); uma instância Evolution por cliente no mesmo VPS
100	Limite de 100 usuários do app Google OAuth; tempo de execução dos crons que percorrem todas as organizações numa chamada só (análise diária, syncs); conexões do Postgres da Evolution
500	Tokens de entrada dominam a conta da IA; VPS único vira gargalo; arquivos de 7 mil linhas tornam cada mudança lenta e arriscada

Dados que faltam para calcular o custo real por cliente

1. Mensagens por mês por cliente (leads e dono) e chamadas de IA por mensagem.
2. Preço vigente por token do modelo usado (conferir na fonte, não de memória).
3. Chamadas de função por mensagem depois da parte 2.
4. Memória e CPU por instância Evolution no VPS, para saber quantos clientes cabem por servidor.
5. Quantas campanhas um cliente cria e diagnostica por mês.
6. Volume de WhatsApp oficial fora da janela gratuita, quando o número oficial for ativado.

10. Experiência do usuário e produto comercial

A experiência pelo WhatsApp foi pensada para leigo e é o ponto forte. O painel web passou por um redesign completo (fases 0 a 4), mas **nenhum cliente consegue usá-lo hoje porque o frontend não está publicado**. A auditoria de usabilidade anterior ao redesign deu nota 22 de 40; a nota depois do redesign não foi medida de novo.

Aspecto	Situação	Evidência
Onboarding	Existe primeiro acesso (`/setup`), implantação pública, checagem do atendente antes de ligar (7 situações de teste). Não há roteiro guiado "conecte WhatsApp → conta de anúncio → primeira campanha" medido com cliente real	`Setup.tsx` , `ImplantacaoPublic.tsx` , `AgentPreLaunchCheck.tsx`
Empresário sem marketing	Bom no WhatsApp: perguntas uma por vez, sem siglas, estimativas sem explicar a origem, "nasce pausada". Na web, o menu caiu de ~60 para 17 itens com explicação de uma linha	regras do assistente, `adminMenu.ts`
Clareza das funcionalidades	Muitas funcionalidades (voz, quiz, cadências, catálogo, comissões) sem uso comprovado; risco de parecer maior do que é	seção 1
Conectar contas de anúncio	Meta: OAuth normal. Google: tela "app não verificado" assusta e o limite é de 100 usuários. O cliente precisa já ter conta de anúncio com pagamento configurado	`google-ads-oauth-start` , `meta-oauth-start`
Primeira campanha	Pelo WhatsApp: cerca de 6 a 10 perguntas, plano em ~80 s (Google, medido em 14/10), confirmação com código. Meta exige foto do dono	seção 2
Uso pelo WhatsApp	Respostas de 28 a 90 s; resumo da manhã, alertas e relatório semanal chegam sozinhos	`admin_agent_summary`

Dependência de suporte técnico (hoje alta)

- Conectar o WhatsApp não oficial (QR) e manter a instância no VPS.
- Instalar o SDK de rastreamento ou o GTM no site do cliente para o Google rastrear vendas.
- Colar parâmetros de URL em anúncios criados fora do sistema.
- Aceitar os termos de conversões otimizadas no Google e verificar o negócio na Meta.
- Criar a chave de IA do cliente (o pool da plataforma está vazio).

O que dificulta a adoção

1. Painel inacessível até publicar o frontend.
2. Sem geração de criativo: o dono precisa ter foto ou vídeo bom.
3. WhatsApp não oficial (risco de banimento) até sair o CNPJ.
4. Google: app não verificado e nenhum caso real de sucesso para mostrar.
5. Nome e marca do produto ainda em aberto.

11. Diagnóstico final

O produto está tecnicamente à frente do que está comercialmente: a lógica de tráfego e atendimento é profunda, mas falta publicar o painel, fechar duas falhas de autorização e provar o sistema com um segundo cliente real antes de cobrar de alguém.

1. Pronto (com uso real)

- Atendente de IA no WhatsApp para leads, com travas de comportamento e passagem para humano.
- Malu pelo WhatsApp: criar, ativar, pausar, mudar verba e remover campanha **Meta**, com confirmação.
- Atribuição de lead de clique-para-WhatsApp ao anúncio; funil por produto; Ganho/Perdido.
- Envio de etapas e vendas à Meta (CAPI) com deduplicação.
- Análise diária automática das campanhas Meta e resumo da manhã no WhatsApp.

2. Funciona parcialmente

- **Google Ads:** criação, gestão e otimização construídas e testadas em campanhas pausadas; nenhuma campanha com cliques reais.
- **Venda → Google:** construído; nenhuma venda aceita.
- **Retorno por campanha:** funciona, mas depende de marcar Ganho com valor.

- **Otimização:** recomenda; não executa sozinha (regras automáticas sem nenhuma cadastrada).

3. Incompleto

- Renovação do token Meta; verificação do app Google; WhatsApp oficial (CNPJ).
- Pool de IA da plataforma (vazio); custo por cliente; limites de gasto de IA por cliente.
- Geração de criativos; Google além de Pesquisa; remarketing Meta (só plano).
- Testes automatizados versionados (só 1 arquivo; testes ficaram em pastas temporárias).

4. Parece existir só na tela

Não encontrei tela sem backend por trás. Encontrei telas cujo backend **nunca rodou com dado real**: card de custo do WhatsApp oficial, regras automáticas de otimização, diagnóstico guiado Google, relatório PDF salvo. Voz, quiz, cadências, catálogo e comissões não foram verificados nesta auditoria.

5. Diferenciais competitivos

1. Gerir anúncios Meta e Google por conversa de WhatsApp, com confirmação e teto de gasto.
2. Metodologia de cursos reais codificada como regra (não como "a IA acha").
3. Ciclo fechado anúncio → conversa → venda → conversão de volta, com funil real do CRM nas regras (qualidade de lead, taxa de conexão).
4. Atendente que move o lead no funil sozinho e avisa o dono sobre lead quente.

6. Maiores riscos técnicos

1. Falhas de autorização entre empresas nas Edge Functions (`outreach-router`; webhook da Evolution sem segredo).
2. VPS único para WhatsApp e MCPs; número não oficial.
3. Arquivos de 6.500 a 7.100 linhas concentrando a lógica central.
4. Configuração fora do Git (crons, segredos do vault).
5. Multi-empresa nunca testado com duas empresas reais.

7. Maiores gargalos comerciais

1. Painel não publicado.
2. Nenhum caso de sucesso Google e um só caso Meta (pequeno, R$ 12/dia).

3. Cada cliente precisa de chave de IA e de muita ajuda na implantação (QR do WhatsApp, rastreamento do site, termos do Google).
4. Sem CNPJ: sem WhatsApp oficial e sem verificação de negócio na Meta.
5. Marca e preço ainda em aberto.

8. O que falta para ter clientes pagantes

Publicar o frontend em domínio próprio; fechar as falhas de autorização; ter IA funcionando para cliente sem chave própria (pool abastecido com limite por plano); rodar um cliente piloto real de ponta a ponta por pelo menos um mês; termo de uso e política de privacidade; plano de contingência do WhatsApp.

9. As 10 prioridades, em ordem

1. **Fechar a autorização:** `outreach-router` conferir usuário e empresa; segredo no webhook da Evolution; código obrigatório para confirmar ações de gasto; auditar toda função que recebe `organization_id` no corpo.
2. **Publicar o frontend** (hospedagem, domínio, `PUBLIC_APP_URL` real).
3. **IA para todo cliente:** abastecer o pool de chaves com limite de uso por plano e registro de custo por organização.
4. **Cliente piloto real** (segunda organização) usando WhatsApp, atendente, Malu e funil por 30 dias.
5. **Voltar a anunciar** (limite de pagamento da Meta) para gerar dado: sem verba, o motor de análise não tem o que analisar.
6. **Fechar o ciclo Google com um caso real:** campanha ativa com cliques, venda aceita na Data Manager, diagnóstico com dado.
7. **WhatsApp:** CNPJ e número oficial, ou separar a Evolution em servidor próprio com monitoramento e plano de troca de número.
8. **Contas de anúncio:** alerta e reconexão do token Meta; verificação do app Google para sair do limite de 100 usuários.
9. **Custo e escala:** juntar as 3 funções por mensagem (parte 2), decidir o plano pago do Supabase, levar crons e segredos para migrations documentadas.
10. **Manutenção:** versionar testes das regras puras e dividir os arquivos gigantes antes de crescer o time ou o número de clientes.